

Guía 2 - Nota técnica

¿Siempre hay que balancear un dataset?

Selección de atributos, sesgo y balanceo de clases

Respuesta central

No siempre hay que balancear. Siempre se debe diagnosticar el desbalance, pero la decisión de balancear depende del problema, la importancia de la clase minoritaria, el costo del error y el comportamiento del modelo.

1. Contexto dentro de la Guía 2

En la Guía 2 se revisa si el dataset tiene condiciones adecuadas antes de pasar al modelamiento supervisado. Por eso se analizan variables, varianza, redundancia, representatividad, sesgo y distribución de la variable objetivo.

El balanceo no es un paso automático. Es una decisión técnica que se toma después de observar si una clase aparece mucho más que otra y si esa diferencia puede afectar el aprendizaje del modelo.

Pregunta de la Guía 2	Cómo se responde	Producto esperado
¿Qué variables aportan?	Revisión de tipos, valores únicos, varianza, redundancia y relación con la variable objetivo.	Tabla de selección de atributos.
¿Hay sesgo o baja representatividad?	Distribuciones por grupo: ciudad, zona, segmento, canal, satisfacción.	Análisis de representación y riesgos.
¿La variable objetivo está balanceada?	Conteo y porcentaje de clases en Abandono.	Diagnóstico y decisión: balancear o no balancear.

2. ¿Qué significa balancear?

Balancear significa ajustar la cantidad de registros de cada clase de la variable objetivo, normalmente para que la clase minoritaria no sea ignorada por el modelo.

Ejemplo

Si Abandono = 0 representa el 95% y Abandono = 1 representa el 5%, un modelo podría aprender a decir casi siempre que nadie abandona. Tendría una accuracy aparentemente alta, pero sería inútil para detectar clientes en riesgo.

3. Diagnóstico: lo que sí se hace siempre

Lo obligatorio no es balancear. Lo obligatorio es revisar la distribución de la variable objetivo y explicar la situación con cantidades y porcentajes.

```
objetivo = 'Abandono'
print(df[objetivo].value_counts())
print(df[objetivo].value_counts(normalize=True).round(4))
```

Ejemplo de interpretación:

Clase	Cantidad	Porcentaje	Interpretación
0 - No abandona	268	74.44%	Clase mayoritaria.
1 - Abandona	92	25.56%	Clase minoritaria, pero no extremadamente baja.

Conclusión del ejemplo

El dataset presenta un desbalance moderado. No se debe balancear automáticamente. Primero se debe evaluar si el modelo detecta correctamente la clase de interés, en este caso Abandono = 1.

4. ¿Cuándo sí conviene balancear?

Situación	Por qué conviene balancear
La clase minoritaria es muy baja.	El modelo puede ignorarla y favorecer la clase mayoritaria.
La clase minoritaria es la más importante para el negocio.	En abandono, fraude o riesgo, detectar el caso positivo suele ser crítico.
El modelo tiene bajo recall para la clase minoritaria.	No está identificando suficientes casos reales de la clase importante.
El costo de no detectar la clase minoritaria es alto.	Perder clientes, no detectar fraude o no anticipar riesgo puede ser costoso.

5. ¿Cuándo no es obligatorio balancear?

Situación	Por qué no necesariamente se balancea
Desbalance leve o moderado.	Puede ser manejable sin modificar la distribución.
El modelo ya detecta bien la clase minoritaria.	Si recall, precision y F1 son adecuados, balancear puede no aportar mejora.
La distribución refleja la realidad del negocio.	Forzar 50/50 puede distorsionar el comportamiento real.
Se usan pesos de clase.	Algunos modelos permiten <code>class_weight=balanced</code> sin duplicar datos.
Hay riesgo de sobreajuste por duplicación.	El oversampling simple puede repetir demasiado los mismos registros.

6. Regla metodológica clave

Regla de oro

Si se decide balancear, el balanceo se aplica solo sobre el conjunto de entrenamiento. No se balancea todo el dataset ni se balancea el conjunto de prueba.

La razón es que el conjunto de prueba debe representar datos no vistos y lo más cercanos posible a la realidad. Si se modifica el conjunto de prueba, la evaluación deja de ser confiable.

Paso	Acción correcta
1	Separar X e y.
2	Dividir en entrenamiento y prueba.
3	Diagnosticar distribución en y_train.
4	Balancear solo X_train, y_train si se justifica.
5	Entrenar el modelo.
6	Evaluar con X_test, y_test sin balancear.

7. Código base para decidir, no para balancear automáticamente

Primero se separan los datos y luego se revisa el desbalance en entrenamiento:

```
from sklearn.model_selection import train_test_split

X = df.drop(columns=['Abandono', 'ID_Cliente'])
y = df['Abandono']

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print(y_train.value_counts())
print(y_train.value_counts(normalize=True).round(4))
```

Si se justifica balancear, puede aplicarse una estrategia solo sobre entrenamiento. Ejemplo con oversampling simple:

```
import pandas as pd
from sklearn.utils import resample

train_data = X_train.copy()
train_data['Abandono'] = y_train

clase_0 = train_data[train_data['Abandono'] == 0]
clase_1 = train_data[train_data['Abandono'] == 1]

clase_1_balanceada = resample(
    clase_1,
    replace=True,
    n_samples=len(clase_0),
    random_state=42
)

train_balanceado = pd.concat([clase_0, clase_1_balanceada])
X_train_balanceado = train_balanceado.drop(columns=['Abandono'])
y_train_balanceado = train_balanceado['Abandono']

print(y_train_balanceado.value_counts())
print(y_train_balanceado.value_counts(normalize=True).round(4))
```

8. Métodos para manejar el desbalance

Método	Qué hace	Cuidado principal
No balancear y medir bien	Mantiene la distribución original y usa métricas adecuadas.	No quedarse solo con accuracy.
class_weight=balanced	Da mayor peso a la clase minoritaria durante el entrenamiento.	Solo aplica en modelos compatibles.
Oversampling	Aumenta registros de la clase minoritaria.	Puede sobreajustar si duplica demasiado.

Undersampling	Reduce registros de la clase mayoritaria.	Puede perder información útil.
SMOTE	Genera ejemplos sintéticos de la clase minoritaria.	Debe usarse con criterio y solo en entrenamiento.

9. Diferencia entre Guía 2 y Guía 3

Elemento	Guía 2	Guía 3
Objetivo	Diagnosticar calidad del dataset antes del modelo.	Entrenar y evaluar modelos supervisados.
Desbalance	Se detecta y se decide si requiere tratamiento.	Se observa su impacto en las métricas del modelo.
Métricas como accuracy, precision, recall, F1	No son el foco principal; se mencionan como transición.	Se calculan después de entrenar y predecir.
F-score de atributos	Puede usarse para selección de atributos.	No es lo mismo que F1-score del modelo.

10. Conclusión para el aprendiz

La decisión de balancear no debe tomarse por costumbre. Primero se diagnostica la distribución de clases, luego se analiza la importancia de la clase minoritaria y finalmente se decide si se balancea, si se usan pesos de clase o si se mantiene la distribución original con métricas adecuadas.

Idea final

Siempre se revisa el desbalance. No siempre se corrige. Se corrige cuando existe evidencia de que puede afectar el aprendizaje del modelo o la detección de la clase importante.

11. Preguntas de comprensión

- ¿Qué diferencia hay entre diagnosticar desbalance y solucionarlo?
- ¿Por qué no se debe balancear el conjunto de prueba?
- ¿Qué riesgo existe si un modelo aprende solo la clase mayoritaria?
- ¿Por qué accuracy puede ser engañosa en datasets desbalanceados?
- ¿En qué casos preferirías `class_weight` en lugar de `oversampling`?